

Early Report: MLSS Exoplanet Habitability Classifier

Bhargav Dave

June 28, 2026

Abstract

This report documents the current state of an MLSS course project that builds a binary classifier for exoplanet habitability labels. The current project is notebook-first and uses a Kepler Object of Interest (KOI) cumulative exoplanet dataset together with separate habitable and non-habitable planet lists. The report explains the data construction, selected feature space, exploratory analysis, preprocessing, model families, and visible results saved in the notebook. It is intentionally explanatory rather than a new modeling stage: no raw data is included, no notebook cells are rerun, and no model results are regenerated. The main interpretive caution is that the target variable `habitable?` is derived from pre-existing labeled lists, so high predictive scores indicate successful reproduction of that label distinction rather than definitive proof of physical or biological habitability.

Contents

1	Project Overview	3
2	Dataset and Provenance	3
3	Target Construction	3
4	Feature Space	4
5	Exploratory Data Analysis	5
6	Outlier Handling	6
7	Train-Test Split and Imputation	7
8	Evaluation Metric	8
9	Logistic Regression Models	8
10	Tree-Based Models	9
11	Support Vector Machines	11
12	Model Comparison	12
13	Interpretation of Results	12
14	Limitations	13

1 Project Overview

This report explains the current state of the MLSS exoplanet classifier notebook. The project is a graduate-level data science and machine learning course project focused on binary classification: predicting whether an exoplanet candidate is labeled habitable or non-habitable.

The current notebook is preserved as a historical course artifact at:

`notebooks/mlss-dacss756-final-project-bdave_v10.ipynb`

The notebook title currently reads “Predicting the Hability of Exoplanets using Machine Learning.” The word “Hability” should eventually be corrected to “Habitability,” but the notebook itself has not been edited for this report.

The modeling goal is to learn a relationship between selected KOI numeric features and a binary target variable called `habitable?`. A value of 1 represents planets from a habitable list, and a value of 0 represents planets from a non-habitable list. This is a supervised classification problem, not an unsupervised discovery task.

2 Dataset and Provenance

The notebook describes the data as Kepler exoplanet and KOI data. It loads a cumulative KOI-style dataset plus two separate labeled files:

- `cumulative-exoplanets.xlsx`
- `habitable_planets_detailed_list.xlsx`
- `non_habitable_planets_confirmed_detailed_list.xlsx`

The raw data files are not included in the clean GitHub repository or in this Overleaf package. This report uses only saved notebook outputs and existing report artifacts.

The visible notebook output shows that the original cumulative table contains 9,564 rows and 141 columns. The notebook keeps only a subset of columns for modeling: `kepoi_name`, 15 numeric KOI features, and later the target variable `habitable?`.

The NASA Exoplanet Archive documentation for the Kepler Object of Interest table describes KOI columns returned through the Exoplanet Archive API, including identifiers and physical/orbital columns such as KOI name, orbital period, planet radius, and stellar parameters [2]. This report does not overclaim provenance beyond what the notebook shows: the notebook uses Kaggle-style input paths and describes the underlying data as Kepler/KOI data.

3 Target Construction

The target variable is `habitable?`. The notebook constructs it in three steps:

1. Assign `habitable? = 1` to every row in the habitable planet list.
2. Assign `habitable? = 0` to every row in the non-habitable confirmed planet list.
3. Concatenate those label tables and merge them onto the filtered KOI feature table by `kepoi_name`.

After this merge, the visible modeling dataset has 2,372 rows and 17 columns. Those columns consist of `kepoi_name`, the 15 numeric modeling features, and `habitable?`.

This construction is central to interpretation. The model is learning to reproduce labels derived from pre-existing habitable and non-habitable lists. It is not directly measuring biological habitability, biosignatures, surface conditions, atmospheric composition, or life. A high score means the model separates these two label-list groups using selected KOI features. It does not prove that the model has discovered physical habitability in a broader scientific sense.

A related limitation is possible label-feature dependence. If the habitable and non-habitable lists were created using variables similar to the predictors, then the model may partly learn the same rules used to construct the labels. That can still be reasonable for a course classifier, but it should be documented carefully.

4 Feature Space

The notebook uses 15 numeric KOI-style features. These features mix planet-candidate properties, orbital geometry, and host-star properties. Table 1 gives plain-language descriptions.

Table 1: Selected modeling features and plain-language descriptions.

Feature	Description
<code>koi_period</code>	Orbital period, usually measured in days. It describes how long the object takes to complete one orbit.
<code>koi_ror</code>	Planet-star radius ratio. It compares the estimated radius of the planet candidate with the radius of its host star.
<code>koi_srho</code>	Fitted stellar density. It is a stellar-density estimate associated with the transit model.
<code>koi_prad</code>	Planetary radius in Earth radii. This is relevant because extremely large objects are less Earth-like.
<code>koi_sma</code>	Semi-major axis in astronomical units. This approximates orbital scale or distance from the host star.
<code>koi_teq</code>	Equilibrium temperature in Kelvin. This is an estimated temperature under simplifying assumptions, not a measured surface temperature.
<code>koi_insol</code>	Insolation flux relative to Earth. It estimates how much stellar energy the planet receives.
<code>koi_dor</code>	Planet-star distance divided by stellar radius. This is a scaled orbital-distance quantity.
<code>koi_count</code>	Count of planets or KOIs associated with the system in the KOI table context.
<code>koi_steff</code>	Stellar effective temperature in Kelvin. This describes the host star, not the planet itself.
<code>koi_slogg</code>	Stellar surface gravity, typically in $\log_{10}(\text{cm/s}^2)$. This is a host-star property.
<code>koi_smet</code>	Stellar metallicity in dex. This is a host-star composition-related estimate.
<code>koi_srad</code>	Stellar radius in solar radii. This is the size of the host star relative to the Sun.

Feature	Description
<code>koi_smass</code>	Stellar mass in solar masses. This is the host-star mass relative to the Sun.
<code>koi_eccen</code>	Orbital eccentricity. It describes how circular or elongated the orbit is.

5 Exploratory Data Analysis

The notebook performs exploratory data analysis after merging labels onto the selected KOI features. The key visible row counts are:

- Original cumulative dataset: 9,564 rows and 141 columns.
- Merged modeling dataset: 2,372 rows and 17 columns.
- Dataset after outlier removal: 2,121 rows.

Visible missingness before processing shows that `kepoi_name`, `koi_period`, `koi_count`, and `habitable?` have 0 missing values, while many other numeric feature columns have 1 missing value each.

Figure 1 shows the saved notebook boxplots for the 15 numeric features before outlier handling. Figure 2 shows the target class distribution.

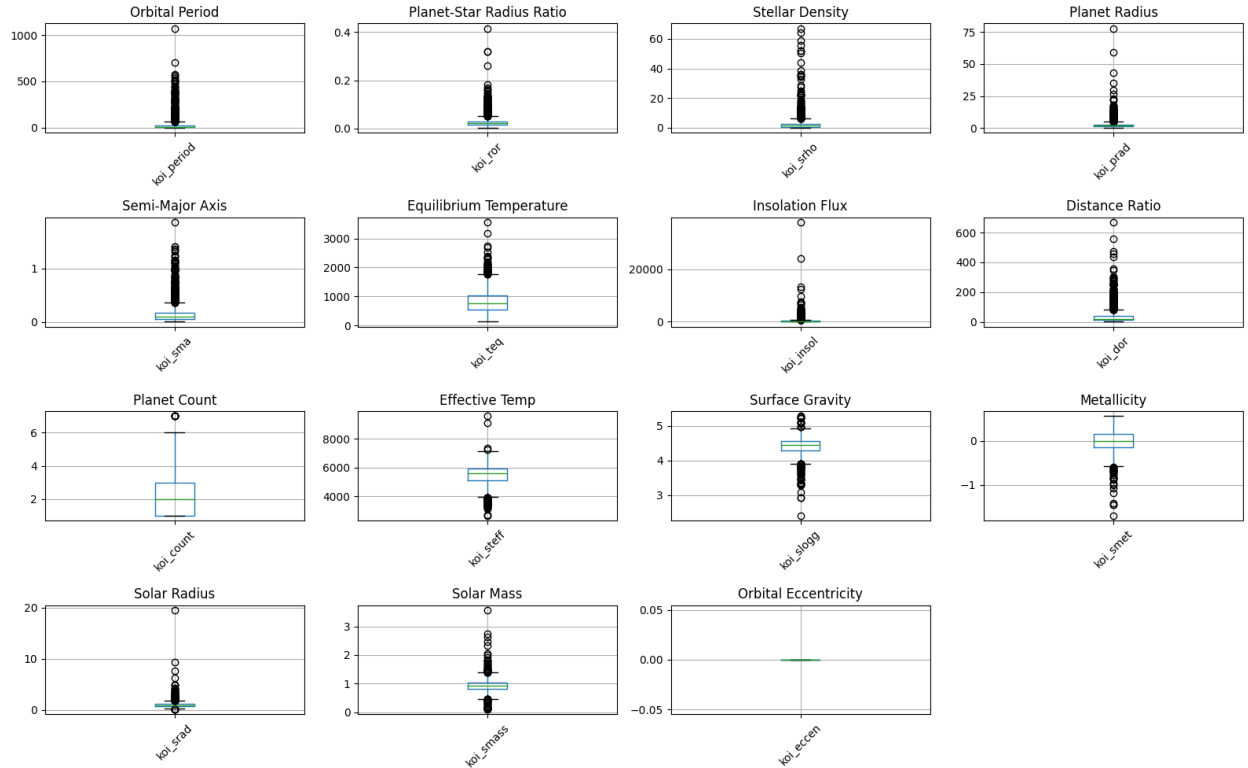


Figure 1: Feature boxplots before outlier removal. The saved notebook output shows substantial spread and potential outliers across several numeric KOI features.

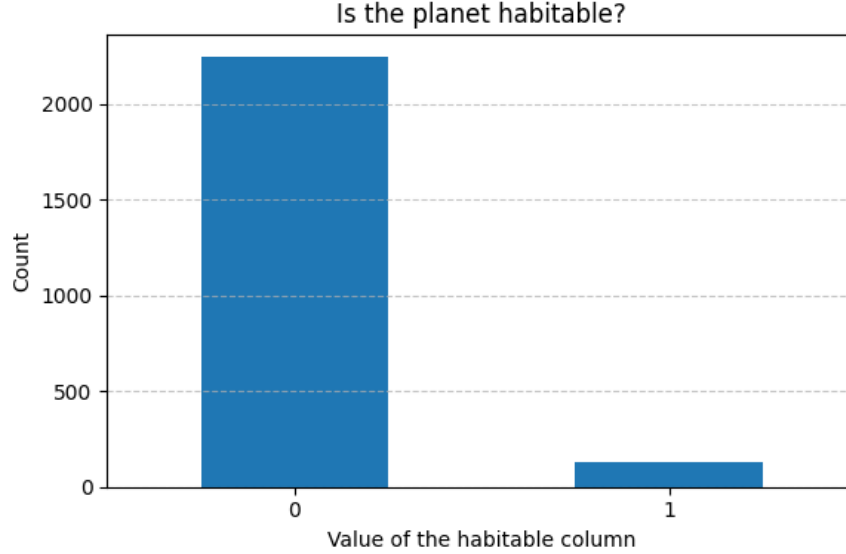


Figure 2: Target class distribution for `habitable?`. The visible notebook output indicates strong class imbalance.

The processed class distribution is highly imbalanced. The visible split sanity check reports approximately 94.86% class 0 and 5.14% class 1 overall. This imbalance explains why the notebook does not rely on accuracy alone.

6 Outlier Handling

The notebook defines an interquartile range (IQR) filtering function. For a chosen column,

$$\text{IQR} = Q_3 - Q_1,$$

$$\text{lower bound} = Q_1 - \text{multiplier} \times \text{IQR}, \quad \text{upper bound} = Q_3 + \text{multiplier} \times \text{IQR}.$$

Rows outside the interval are removed. The notebook applies this filtering to two features:

- `koi_smass`, using IQR multiplier 1.5.
- `koi_prad`, using IQR multiplier 2.

The visible row counts are:

- Before `koi_smass` filtering: 2,372 rows.
- After `koi_smass` filtering: 2,276 rows.
- After `koi_prad` filtering: 2,121 rows.

This choice should be documented carefully. Outlier removal can strongly affect rare classes, and the habitable class is only about 5% of the processed data. Removing rows based on stellar mass or planet radius may remove physically unusual but scientifically meaningful cases. A future workflow should quantify whether filtering disproportionately removes class 1.

7 Train-Test Split and Imputation

The notebook separates features and target after outlier filtering:

- `X`: the 15 numeric features.
- `y`: `habitable?`.

It then performs an 80/20 train-test split with `test_size=0.2`, `stratify=y`, and `random_state=42`. Stratification is important because the positive class is small. Without stratification, a random split could place too few habitable-class examples in either train or test.

The visible classification reports show the test set has 403 class 0 examples and 22 class 1 examples. That small positive-class test support is a major interpretation caveat: one or two changed predictions can noticeably affect positive-class precision, recall, and F1.

The notebook uses median imputation. It fits `SimpleImputer(strategy='median')` on `X_train` and applies it to `X_test`. This is better than fitting imputation before the split, because the test set does not influence the training-set medians.

However, future refactoring should move imputation inside scikit-learn `Pipeline` and, if needed, `ColumnTransformer` objects [7, 4]. In the current notebook, imputation is fit before cross-validation over `X_train`, which means median values are computed from the full training set before internal CV folds are made. Because missingness is tiny here, the practical effect may be small, but the leakage-safe pattern is to fit preprocessing inside each CV fold.

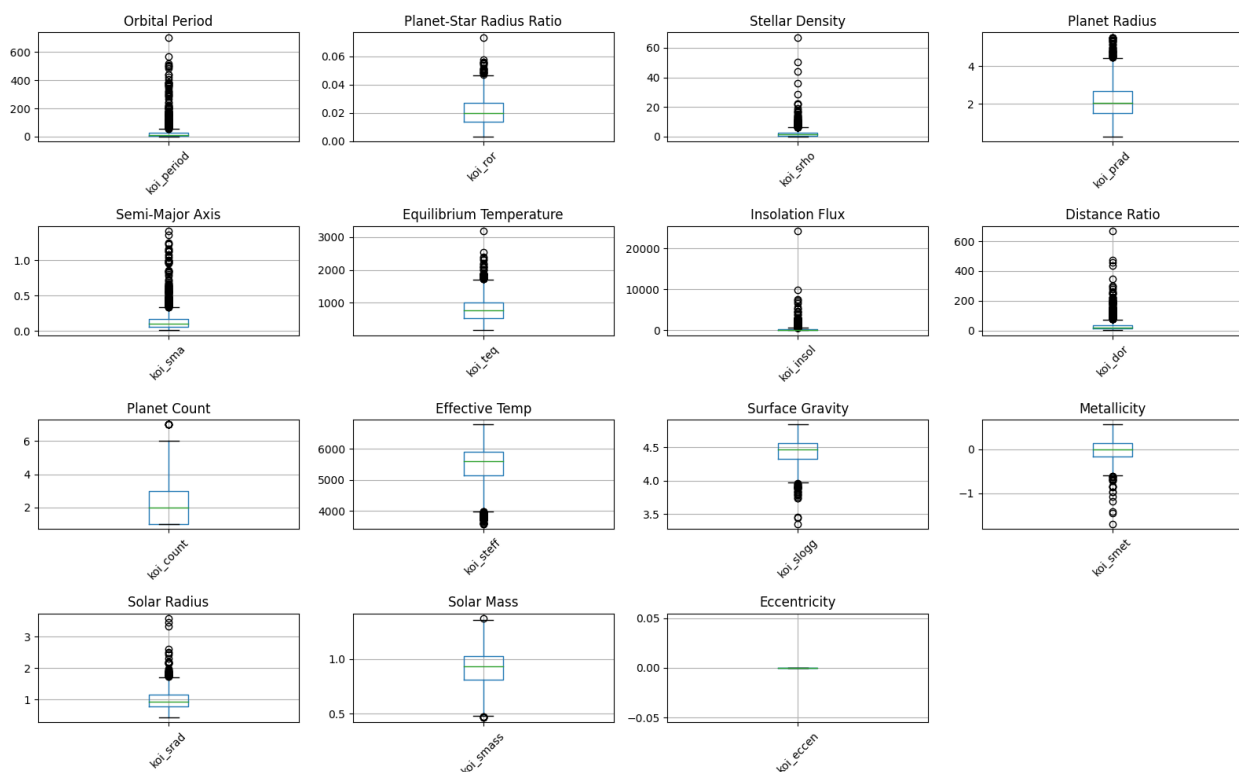


Figure 3: Training feature boxplots after the train-test split, as saved in the notebook output.

8 Evaluation Metric

The notebook uses macro F1 as the main model-selection metric. For binary classification:

- Accuracy is the fraction of all predictions that are correct.
- Precision asks: among predicted positives, how many are truly positive?
- Recall asks: among actual positives, how many were found?
- F1 is the harmonic mean of precision and recall.

The scikit-learn documentation defines F1 as a balance between precision and recall [5]. F1 is useful when the positive class is important and class distribution is imbalanced.

Macro averaging computes the metric separately for each class, then averages the class-level values. This gives class 0 and class 1 equal weight, even though class 1 is much smaller. A classifier can achieve high accuracy by mostly predicting the majority class, but macro F1 penalizes poor minority-class performance.

The notebook uses `GridSearchCV` with 5-fold cross-validation and `scoring='f1_macro'`. Cross-validation estimates model performance across multiple train/validation splits, and grid search tries parameter combinations to select the best setting under the chosen metric [3, 6].

9 Logistic Regression Models

Binary logistic regression models the probability that an example belongs to class 1 rather than class 0. It learns a linear decision boundary in the feature space. In this project, the inputs are numeric exoplanet and stellar features, and the output is the classification target `habitable?`.

Because logistic regression is sensitive to feature scale, the notebook uses `StandardScaler` inside scikit-learn pipelines for the logistic models. Scaling matters because features such as orbital period, stellar temperature, and metallicity exist on very different numeric ranges.

The notebook evaluates three regularized logistic regression variants:

- L1 regularization, also called lasso-style regularization. It can push some coefficients toward exactly zero.
- L2 regularization, also called ridge-style regularization. It shrinks coefficients but usually does not set them exactly to zero.
- Elastic Net regularization, which combines L1 and L2 penalties.

The notebook searches over `C`, where `C` is the inverse of regularization strength. Smaller `C` means stronger regularization; larger `C` means weaker regularization.

Visible logistic regression results are:

- L1 Logistic Regression: CV macro F1 = 0.9409.
- L2 Logistic Regression: CV macro F1 = 0.9384.
- Elastic Net Logistic Regression: CV macro F1 = 0.9419 and test macro F1 = 0.9648.

The final comparison table includes Elastic Net logistic regression, not the L1 and L2 variants. The Elastic Net model ties for the best visible test macro F1 in the final comparison.

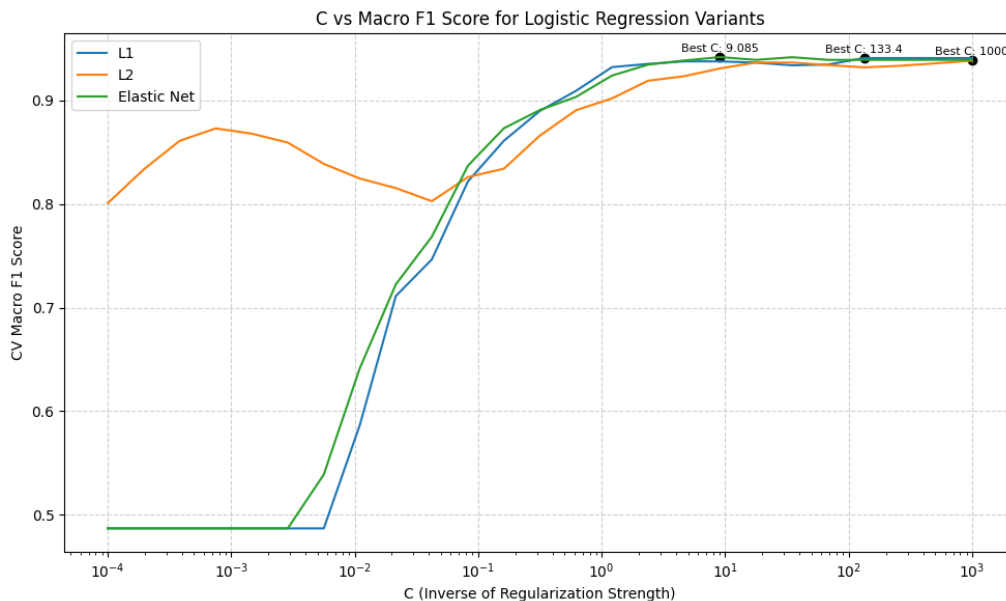


Figure 4: Saved logistic regression cross-validation curve from the notebook.

10 Tree-Based Models

Tree-based models split the feature space into regions using decision rules. A single decision tree can be intuitive, but it often overfits. Ensemble tree methods combine many trees to improve performance.

The notebook includes Random Forest, Gradient Boosting, and XGBoost.

Random Forest. Random Forest builds many decision trees and combines their predictions. It usually reduces variance relative to a single tree by using randomness in samples and/or selected features.

Gradient Boosting. Gradient Boosting builds trees sequentially. Each new tree attempts to improve the current ensemble by focusing on remaining errors.

XGBoost. XGBoost is an optimized gradient-boosting framework often used for tabular data. The original XGBoost paper describes it as a scalable tree boosting system [1], and the official documentation describes the boosted-tree model formulation [8].

Visible tree-based results are:

- Random Forest: CV macro F1 = 0.9718 and test macro F1 = 0.9540.
- Gradient Boosting / “Random Forest With GBM”: CV macro F1 = 0.9720 and test macro F1 = 0.9540.
- XGBoost: CV macro F1 = 0.9647 and test macro F1 = 0.9413.

The notebook includes impurity-based feature importance plots. It reports `koi_teq` as the most important Random Forest feature and `koi_insol` as the most important Gradient Boosting feature.

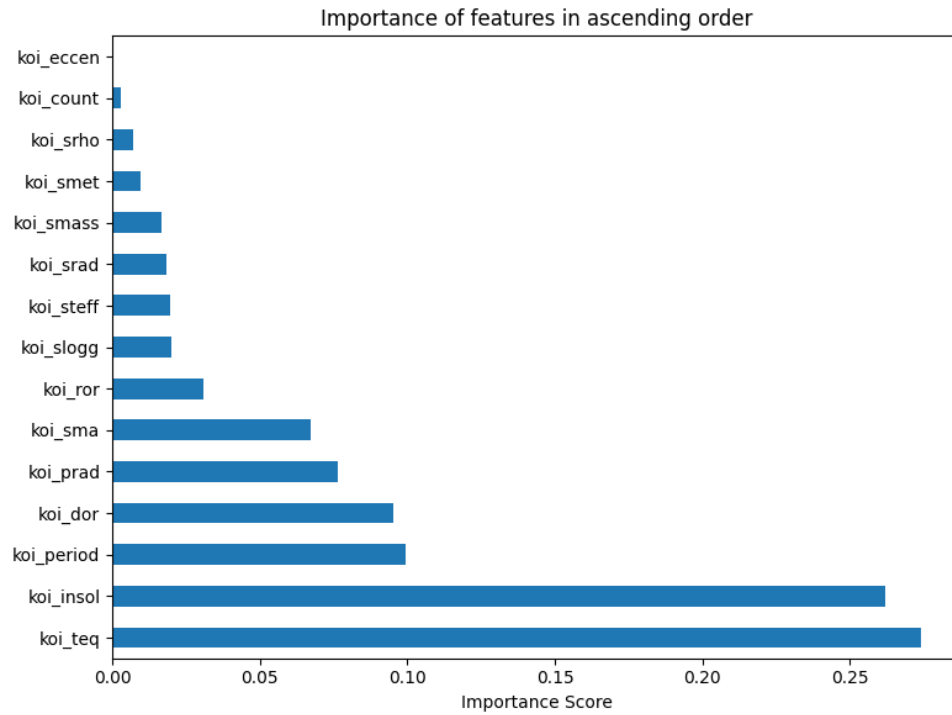


Figure 5: Random Forest feature importance from the saved notebook output. The visible text identifies `koi_teq` as the most important feature.

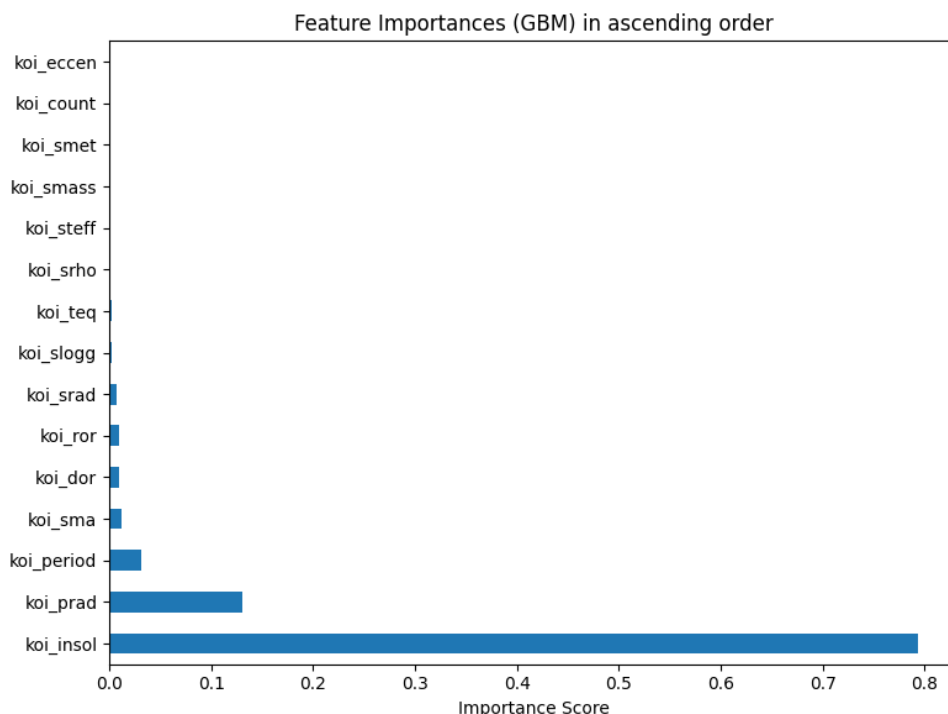


Figure 6: Gradient Boosting feature importance from the saved notebook output. The visible text identifies `koi_insol` as the most important feature.

These importances should be interpreted cautiously. Impurity-based feature importance can favor variables with certain distributions or many possible split points, and correlated features can share or distort importance. Future work should add permutation importance as a more model-agnostic check.

11 Support Vector Machines

Support vector machines classify examples by finding a boundary that separates classes with a large margin. The support vectors are the training points closest to the decision boundary; they help define the margin.

The notebook evaluates three SVM kernels:

- Linear kernel: learns a linear boundary.
- Polynomial kernel: allows curved boundaries based on polynomial feature interactions.
- RBF kernel: allows flexible nonlinear boundaries based on radial similarity.

The key hyperparameters include:

- `C`: controls the tradeoff between margin size and training errors.
- `gamma`: controls locality of influence for kernels such as RBF.
- `degree`: controls polynomial complexity for the polynomial kernel.

The notebook uses `StandardScaler` for SVM models because SVM distances and margins depend on feature magnitude.

Visible SVM results are:

- SVM Linear: CV macro F1 = 0.9419 and test macro F1 = 0.9540.
- SVM Polynomial: CV macro F1 = 0.9399 and test macro F1 = 0.9338.
- SVM RBF: CV macro F1 = 0.9421 and test macro F1 = 0.9648.

The RBF SVM ties Elastic Net logistic regression for the best visible test macro F1 in the final comparison.

12 Model Comparison

Table 2 reproduces the final visible model comparison table. These values are also stored in the report package under `tables/model_comparison.csv` and `tables/model_comparison.md`.

Table 2: Visible final model comparison from the notebook.

Model	CV macro F1	Test macro F1
Logistic Regression (Elastic Net)	0.9419	0.9648
Random Forest	0.9718	0.9540
Gradient Boosting / “Random Forest With GBM”	0.9720	0.9540
XGBoost	0.9647	0.9413
SVM Linear	0.9419	0.9540
SVM Polynomial	0.9399	0.9338
SVM RBF	0.9421	0.9648

The notebook label “Random Forest With GBM” appears to refer to the Gradient Boosting model implemented with `GradientBoostingClassifier`. This report uses the combined label “Gradient Boosting / Random Forest With GBM” to preserve the visible notebook wording while clarifying the method.

13 Interpretation of Results

The best visible test macro F1 values are:

- Logistic Regression Elastic Net: 0.9648.
- SVM RBF: 0.9648.

The best visible cross-validated macro F1 values are:

- Gradient Boosting: 0.9720.
- Random Forest: 0.9718.

This creates a useful interpretation question. The tree-based models have stronger CV macro F1 scores, but Elastic Net logistic regression and RBF SVM have the highest visible test macro F1. That does not automatically mean the latter are truly better in general. The test set contains only 22 habitable-class cases, so the test macro F1 estimate for class 1 is high-variance. With such small positive test support, a small number of changed predictions can materially alter the result.

The repeated use of the same test set for final model reporting should also be interpreted with care. That is common in course notebooks, but if modeling decisions are influenced by test results, the test set starts to behave like part of the model-selection loop. A stronger future workflow would preserve a final holdout set or use nested cross-validation if rigorous model-selection claims are required.

High scores may indicate that the selected KOI features strongly reproduce the distinction in the pre-labeled habitable and non-habitable lists. That is a valid supervised-learning result, but it should not be stated as proof of definitive physical habitability. The model is learning labels, and the meaning of those labels depends on how the label files were originally constructed.

14 Limitations

The current project has several limitations that should be acknowledged.

- The target is derived from two pre-labeled files and is not a direct biological habitability measurement.
- The processed data is about 94.86% class 0 and 5.14% class 1.
- The visible test set has only 22 class 1 examples.
- If the label lists were created using features related to `koi_teq`, `koi_insol`, `koi_prad`, or similar variables, the model may learn label-generation logic rather than independent habitability evidence.
- Imputation is fit after the train-test split, which is good, but it should be inside pipelines for cross-validation safety.
- Outlier filtering happens before train-test split and before imputation, so future work should document how it affects class balance.
- The notebook uses Kaggle-specific paths such as `/kaggle/input/...`
- Raw data acquisition is not scripted.
- There is no root README yet.
- There is no requirements file yet.
- There are no automated tests yet.
- The workflow is notebook-only and not yet modular.
- The title typo “Hability” should eventually be corrected to “Habitability.”
- Some labels should be standardized, such as “Random Forest With GBM” versus “Gradient Boosting.”

15 Roadmap for Improvement

The enhanced version should preserve the original notebook as a course artifact, then build a clean reproducible workflow around it. Recommended improvements include:

- Add a root README explaining the project goal, setup, data access, reproducible commands, and scientific caveats.
- Add `requirements.txt` or `pyproject.toml`.
- Add local path handling so the project does not depend on Kaggle-only paths.
- Add a config-driven workflow for paths, feature lists, split settings, model grids, and output locations.
- Move preprocessing into scikit-learn `Pipeline` and `ColumnTransformer` objects.
- Keep imputation, scaling, and model fitting leakage-safe inside cross-validation.
- Add a `DummyClassifier` baseline.
- Use bounded `GridSearchCV` or `RandomizedSearchCV` so searches are reproducible and not unnecessarily expensive.
- Reproduce existing notebook models before introducing any new modeling methods.
- Save model comparison artifacts as CSV, Markdown, and figures.
- Add confusion matrices, ROC curves, and precision-recall curves.
- Add permutation importance for model interpretation.
- Create a scientific caveats document explaining what the `habitable?` label does and does not mean.
- Add lightweight tests for schema expectations, feature selection, target construction, split stratification, and output creation.
- Add CLI scripts for data preparation, training, and evaluation.
- Define an optional model persistence policy, keeping model binaries out of Git.

The immediate next engineering stage should be reproducibility and structure, not new modeling complexity.

References

- [1] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. Association for Computing Machinery, 2016.
- [2] NASA Exoplanet Archive. *Data Columns in the Kepler Objects of Interest Table*. NASA Exoplanet Archive, IPAC, Caltech. Official KOI table column documentation. Accessed 2026-06-28.
- [3] scikit-learn developers. *Cross-validation: evaluating estimator performance*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [4] scikit-learn developers. *sklearn.compose.ColumnTransformer*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [5] scikit-learn developers. *sklearn.metrics.f1_score*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [6] scikit-learn developers. *sklearn.model_selection.GridSearchCV*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [7] scikit-learn developers. *sklearn.pipeline.Pipeline*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [8] XGBoost developers. *Introduction to Boosted Trees*. XGBoost Documentation. Official documentation. Accessed 2026-06-28.